

Emotional AI Mirror

Final Project Report

Template 4.1: Orchestrating AI models to achieve a goal

CM3020 Artificial Intelligence - Project Idea 1, Template 4.1

[https://github.com/leonardo-bracci/CS Final Project](https://github.com/leonardo-bracci/CS_Final_Project)

Word count: 7322 (excluding headings, figure and table captions, tables and references)

Table of Contents

1. Introduction – word count (511).....	4
Project goal, aims and objectives.....	4
2. Literature Review – word count (1735).....	5
2.1 Multimodal emotion-related systems.....	5
2.2 From emotion classification to psychological inference.....	5
2.3 Multimodality, context and model limitations.....	5
2.4 Application context: support, agency and privacy.....	6
2.5 Synthesis and implications for this project.....	6
2.6 Techniques and methods.....	7
OpenCV, Computer Vision Library.....	7
Whisper / faster-whisper, Speech-to-Text Model.....	7
Sentiment / Text Emotion Detection.....	8
Ollama, Local Language Model Framework.....	8
2.7 Study cases and research evidence.....	8
2.8 Scientific validity of facial-emotion inference.....	9
3. Design – word count (1385).....	9
Domain.....	9
Design justification.....	10
Fully local processing.....	10
Multiple models across different domains.....	10
Interactive conversation.....	10
Safety signposting and scope boundary.....	10
Inclusive design.....	11
Ethical safeguards, privacy management and crisis boundary.....	11
Structure of the project.....	12
4. Implementation – word count (1281).....	13
Orchestration overview.....	13
Stage 1: Facial emotion recognition (DeepFace).....	13
OASIS integration and picture-in-picture display.....	16
Stage 2: Speech-to-text (Whisper / faster-whisper).....	16
Stage 3: Sentiment / text-emotion analysis.....	17
Stage 4: Local language model (Ollama).....	17
Integration test.....	19
Session flow and user interaction.....	20
5. Evaluation – word count (1982).....	22
Testing methodology.....	22
Stage 1 results: Facial emotion recognition.....	23
DeepFace vs. FER comparison.....	23
Stage 2 results: Speech-to-text.....	25
Stage 3 results: Sentiment / text-emotion analysis.....	26
Stage 4 results: Local language model (Ollama).....	28
End-to-end pipeline timing.....	30
Session logging.....	31

Unit testing.....	32
User testing.....	33
Seek-help redesign and re-evaluation.....	34
Inclusive design and demographic limitations.....	34
Evaluation against project objectives.....	35
6. Conclusion – word count (428).....	36
References.....	36

1. Introduction – word count (511)

This project is a privacy-focused proof of concept for emotional self-reflection. It combines facial-emotion classification, speech transcription, text-emotion analysis and a local language model in a single guided session. Users view an emotional stimulus while their facial responses are sampled, answer three spoken questions, and receive a personalised, non-diagnostic reflection based on the resulting signals. All processing is designed to remain on the user's device. The prototype is explicitly not a diagnostic or therapeutic system: its outputs are hypotheses for reflection, not reliable measurements of internal psychological states or substitutes for professional care.

The project uses Template 4.1: Orchestrating AI models to achieve a goal (CM3020 Artificial Intelligence, Project Idea 1, section 4.1).

Project goal, aims and objectives

The goal is to demonstrate whether heterogeneous, locally executed AI components can be orchestrated into a usable and privacy-preserving reflection workflow without presenting model outputs as clinically valid psychological measurements.

The project has four aims: implement a complete consent-first local pipeline; compare candidate models using reproducible component-level measurements; evaluate end-to-end robustness, latency and user-facing behaviour; and identify safety, privacy and validity limitations that prevent the prototype from being treated as a diagnostic tool.

The report uses measurable technical objectives rather than a claim of psychological accuracy: complete three end-to-end runs; retain only locally executable components; report exact categorical results and latency for fixed benchmark inputs; pass the project's own automated tests; and collect early user feedback on how clear the output was, how accurate it felt, and whether people would use the system again.

Most people struggle to understand their own emotions. Professional support is expensive and not always accessible. Mental disorders constitute five of the top ten causes of disability, and their total cost across Europe is estimated at more than €600 billion annually (Laidi et al., 2023). This tool targets two groups: those who cannot afford professional therapy, and those who experience anxiety or stigma when discussing mental health with strangers.

Similar technology already exists but serves different purposes. Affectiva, a commercial product born out of MIT Media Lab, uses computer vision to classify facial expressions into emotion categories in real time. However it is built for corporate use, its primary application is measuring consumer reactions to advertisements. The insights go to the advertiser, not the person being analysed. It is expensive, closed source, and inaccessible to individuals.

A more recent example is Ghatoray and Li (2025), from Brunel University London, who combined facial emotion recognition, speech-to-text, and sentiment analysis, feeding everything into a language model, a pipeline technically very similar to this project. However their system only processes pre-recorded video, is not interactive, and uses GPT-4o, a paid cloud service. The authors themselves flagged this as a privacy concern.

This project addresses both gaps: it uses a live webcam feed instead of pre-recorded video, and runs the language model entirely locally using Ollama. Rather than producing a static report for a researcher or feeding data to an advertiser, the system opens an interactive conversation with the user, putting the insight back in the hands of the person it came from.

2. Literature Review – word count (1735)

2.1 Multimodal emotion-related systems

Existing systems show that emotional signals can be picked up from faces, voices and text, but they vary widely in what they are built for, how users interact with them, and how strong the evidence behind them is. Affectiva represents a commercially mature approach: it combines facial, vocal and other behavioural signals for real-time analysis, primarily for organisational applications such as advertising and customer research (Affectiva, 2016; Miller, 2017). Ghatoray and Li (2025) provide a closer technical comparison to this project by combining facial emotion recognition and text sentiment before using a language model to produce higher level insights. Their work supports the case for combining several signal types, but it runs on pre-recorded research videos and relies on GPT-4o, a cloud service. It therefore shows nothing about whether a live, locally run system can safely draw conclusions about a user's psychological state. These examples show that combining models in this way is technically workable, while making one point clear: successfully merging signals is not the same as measuring emotion accurately.

2.2 From emotion classification to psychological inference

The key scientific question is what an emotion label actually tells us. Barrett et al. (2019) reviewed more than 1,000 studies and challenged the assumption that a given facial expression reliably points to a specific inner emotion. They identify three problems: the same emotion can appear on the face in different ways, the same expression can occur in different emotional states, and the link between the two shifts across people, situations and cultures. This limits any system, including Affectiva and this project, that treats a facial label as what a person actually feels. It matters especially here because DeepFace labels one frame at a time, and those labels are then handed to a language model that turns them into smooth, readable prose. This project treats facial and text-emotion outputs as rough signals to reflect on, not as measurements or diagnoses of what someone feels inside.

2.3 Multimodality, context and model limitations

The literature also suggests that using only one signal is limiting. Affectiva's multimodal design and Ghatoray and Li's mix of facial and text data both show the practical value of watching several channels at once. This project therefore combines facial movement, speech, text emotion and language generation to give the user something richer to reflect on than any single classifier could offer. But combining signals should be seen as a way of cross-checking, not as proof. If the underlying models are biased, missing context or unsure, merging their outputs makes the story sound more convincing without making it any more accurate. This is why the project tests each component on its own, then tests the whole pipeline and records where it fails. The fact that DeepFace and the chosen text-emotion

model share the same seven emotion labels is likewise a practical convenience that makes comparison possible.

Multimodal systems are usually classified by where signals are combined: early, late or hybrid fusion (Baltrušaitis et al., 2019; Poria et al., 2017). This project uses late fusion: each model gives its own label, and the language model combines them. Model confidence scores are also often overconfident (Guo et al., 2017), so the thresholds used here are practical cut-offs, not probabilities. Finally, HCI research suggests that emotion technology should help people interpret their own feelings rather than measure them (Boehner et al., 2007), which supports presenting the output as a prompt for reflection.

2.4 Application context: support, agency and privacy

Application context changes the requirements for an emotion-related system. Abby AI provides a useful contrast: it prepares a structured patient summary for a clinician, while the professional remains responsible for interpretation and decision-making (Abby Health, 2026). The present project removes that professional intermediary and presents the generated reflection directly to the user, so it cannot inherit the same safeguard of clinician oversight. This makes scope, consent and communication of uncertainty central design requirements rather than optional additions. Privacy is similarly consequential. Commercial systems such as Affectiva demonstrate the feasibility of large-scale emotion analysis but are oriented towards organisational users, whereas this project deliberately keeps processing local so that sensitive webcam, audio and generated-profile data are not transmitted to a cloud service. These contrasting applications therefore lead to a narrower project position: a local, interactive proof of concept for self-reflection, rather than an autonomous mental-health assessment tool.

2.5 Synthesis and implications for this project

Taken together, the literature supports three conclusions that shape this project. First, multimodal AI pipelines work in practice and can combine signals that complement each other, as both commercial systems and recent research show. Second, the scientific evidence does not support treating facial expressions as direct, reliable measures of inner emotion, which sets a hard limit on what this prototype can claim (Barrett et al., 2019). Third, a system used directly by individuals around their own wellbeing needs stronger safeguards than one built for advertising or for supporting clinicians. These conclusions shape the project's design, namely local processing of all signals, explicit consent, non-diagnostic wording and clear signposting to professional help, and they also set the limit on what counts as success. Table 1 summarises this comparison. The right question to ask is therefore not whether the system can read a person's psychological state, but whether it can combine several imperfect signals into a clear, private reflection process that is honest about its own uncertainty and limits.

Approach	What it demonstrates	Key limitation	Project implication
Abby AI	AI-generated structured profiles	Clinician remains responsible	Direct-to-user output needs stronger scope and safety framing
Affectiva	Real-time multimodal analysis	Commercial context and uncertain psychological inference	Supports feasibility, not validity; motivates local processing
Ghatoray & Li (2025)	Facial + text + LLM pipeline	Pre-recorded video and cloud GPT-4o	Supports multimodal architecture while motivating local interactive design
Emotional AI Mirror	Local multimodal reflection workflow	No validated ground truth; inherited model bias	Proof of concept for self-reflection, not diagnosis

Table 1. Comparison of the reviewed approaches against the position taken by this project.

2.6 Techniques and methods

This project combines several pre-trained models to achieve a common goal across four domains: vision, audio, text, and language generation. The library and model choices below were explored starting from more demanding options and downgraded to lighter frameworks where hardware constraints required it.

OpenCV, Computer Vision Library

OpenCV is an open-source library for detecting faces and tracking movement in videos. The library is cross-platform and licensed as free and open-source software (OpenCV, 2026), making it well suited to this project.

Whisper / faster-whisper, Speech-to-Text Model

For speech recognition the project uses Whisper, an open-source speech recognition model created by OpenAI, trained on 680,000 hours of multilingual audio covering 96 languages besides English (Radford et al., 2022). In practice the project uses faster-whisper (SYSTRAN, n.d.), a reimplementaion reported to run up to four times faster at equivalent accuracy, which matters given the target hardware has no dedicated GPU.

Whisper transcribes what the user says during the session; the transcribed text is then analysed for sentiment and emotional tone.

faster-whisper was benchmarked directly against the original openai-whisper on the same audio file (both at the tiny model size, see Evaluation, Stage 2 results). faster-whisper was confirmed faster on transcription and, on this test file, more accurate: openai-whisper mis-transcribed one word ('microphone' as 'Micah Cup') where faster-whisper transcribed the file correctly. Vosk was also considered as a lighter-weight fallback option but was not directly benchmarked, since the openai-whisper comparison already demonstrates the accuracy/performance case for faster-whisper against the model it is built on.

Sentiment / Text Emotion Detection

To detect emotion in text, the transcribed speech from Whisper is fed to a pre-trained model to identify emotional tone and sentiment. These models use deep learning to classify text as positive, negative, or neutral, or to detect more nuanced emotions like frustration, joy, or sadness (Hugging Face, 2024).

Three candidates were benchmarked on a fixed set of test transcripts: VADER, a rule-based analyser (Hutto & Gilbert, 2014); tabularisai/multilingual-sentiment-analysis (tabularisai, 2024); and j-hartmann/emotion-english-distilroberta-base. j-hartmann (Hartmann, 2022) was selected, both for its higher accuracy on the test set and because it shares the same seven-emotion label space as the DeepFace facial-emotion model, allowing facial and verbal emotion to be compared directly. Full results are given in the Evaluation chapter.

Ollama, Local Language Model Framework

For the interactive component the project uses Ollama, a framework that allows large language models to run locally without an internet connection or cloud services (Ollama, 2024). Ollama generates the final non-diagnostic reflection and personalised recommendations from the combined facial emotion, transcribed speech, and sentiment data.

llama3.2 (Meta, 2024), approximately 2GB, was used for initial integration, chosen as a small widely-used Ollama model and confirmed to run on CPU-only consumer hardware without a dedicated GPU.

llama3.2 and qwen2.5:3b (Qwen Team, 2024) were benchmarked on an identical sample prompt (see Evaluation, Table 8). qwen2.5:3b was notably faster on both time-to-first-token and total generation time, but its reply did not include a recommendation to seek professional help despite the sample containing a fear signal comparable to earlier tests where llama3.2 reliably included one. llama3.2 was retained as the primary model; since the seek-help redesign, this omission no longer affects safety (see Evaluation).

2.7 Study cases and research evidence

To assess the risks of AI-generated mental-health conclusions this project draws on a study by Alyssa Ezell, conducted at Widener University, testing whether six AI chatbots, three general-use (ChatGPT, Gemini, Claude) and three therapist-specific (Pi, Panda, Abby), could accurately diagnose mental health conditions and create treatment plans (Ezell, 2026).

General-use chatbots provided diagnoses in 100% of cases, but only 36.7% were completely correct; in 40% of cases, chatbots gave both correct and incorrect diagnoses simultaneously (Ezell, 2026). Therapist-specific chatbots were more cautious, providing diagnoses only 53.3% of the time. For treatment planning, general-use chatbots recommended plans in 93.3% of cases, most commonly Cognitive Behavioral Therapy, psychiatric medication, and social support, but these recommendations were very generic (Ezell, 2026).

Ezell's research demonstrates that AI can provide mental health information, but current systems struggle with accuracy, personalisation, and transparency. This project does not claim to resolve the accuracy problem Ezell identifies. It responds instead by narrowing the claim: outputs are framed as non-diagnostic reflection rather than assessment, and processing is kept local so that the transparency and privacy concerns raised in that work are not compounded.

2.8 Scientific validity of facial-emotion inference

As established in Section 2.2, DeepFace's frame-by-frame labels should not be read as a direct measure of internal emotional state, and passing them through a language model only adds fluency, not validity. This has a direct consequence for how the evaluation in this report should be read: agreement between two facial classifiers is not evidence that either is psychologically correct, and a user's agreement with a generated reflection is not a clinical ground truth. The appropriate claim for this submission is therefore proof of concept, the system demonstrates a technically functioning multimodal pipeline and exposes where robustness and validity remain unresolved, rather than demonstrating accurate emotional measurement.

3. Design – word count (1385)

Domain

Laidi et al. (2023) report that mental disorders constitute five of the top ten causes of disability, with a total cost across Europe exceeding 4% of GDP, more than €600 billion annually. They also note that these conditions often begin early in life, with significant impacts on work productivity and quality of life.

The primary users of this product are individuals aged 18-45 seeking mental health support in Europe. The scale of the problem is considerable: in France alone, Laidi et al. (2023) estimate the annual cost of mental health at €163 billion in 2018, broken down into direct healthcare costs (14%), productivity loss (27%), and loss of quality of life (51%). These costs reflect both the prevalence of mental health conditions and barriers to access, with anxiety and depressive disorders being the most common conditions, affecting millions of people across the continent.

Affordability is not the only issue: discomfort with human interaction is also a critical barrier. While people generally prefer human doctors to AI systems, there is evidence that a fully automated chatbot can lower the perceived barrier to disclosing personal difficulties, because users feel less exposed to judgement than with another person (Lee et al., 2024);

some individuals may therefore engage with AI-based tools when human interaction feels inaccessible or uncomfortable (Riedl et al., 2024).

This tool is for the two user groups described in the Introduction. It does not diagnose; it points users towards professional help when needed.

Design justification

The design choices in this project are based on the specific needs identified in the domain and user analysis: privacy, affordability, accessibility, and responsible care-seeking.

Fully local processing

To address privacy concerns that are especially important in mental health contexts, the entire system runs locally, with no internet connection or cloud services. Riedl et al. (2024) found that perceived privacy invasion was higher in AI-based interactions, and higher still in psychiatry than in other medical fields, where patients fear data breaches when discussing personal conditions. By keeping all emotional data on the user's machine, the system removes this barrier entirely. This is also what separates the project from commercial products like Affectiva, which send insights to third parties rather than the user.

Multiple models across different domains

The system combines facial emotion recognition, speech transcription, sentiment analysis, and a local language model. This is because people often find it hard to put their feelings into words, and what they say does not always match what their face shows. Using more than one source of information gives a fuller picture than any single model could. Recent work in the field follows the same logic: Ghatoray and Li (2025), for example, combined facial emotion recognition with text sentiment because no single model captured the full emotional picture on its own.

Interactive conversation

After the session signals have been processed, the system opens a dialogue through the local language model instead of producing a static diagnostic label. This matters because the target users are often approaching mental health for the first time and need guidance, not just a label. An interactive format lets them ask questions, explore recommendations, and feel heard. Lee et al. (2024) found that AI chatbots can foster rapport and self-disclosure, which is valuable for users who feel anxious or embarrassed talking to a stranger.

Safety signposting and scope boundary

The system is not designed to replace therapy. Every session ends with professional-support resources, plus a direct recommendation when the user's own words indicate distress, computed in code rather than by the language model. The evidence is clear that people prefer and benefit from human care (Riedl et al., 2024), and an AI tool that masks the need for real treatment would be harmful.

Inclusive design

The system needs no subscription, no referral from a clinician and no paid appointment. It runs entirely on a personal laptop, and nothing is sent off the device. For someone who cannot afford therapy, or who feels uncomfortable discussing mental health with another person, being able to reflect privately at no ongoing cost is itself an argument for inclusion, and it serves the two user groups identified in the Domain section above.

Against this, every pre-trained model in the pipeline carries demographic bias that this project inherits rather than fixes. Facial-analysis models are well documented to work unevenly across skin tone and gender: the Gender Shades audit found commercial gender-classification error rates below 1% for lighter-skinned men, against over 30% for darker-skinned women (Buolamwini & Gebru, 2018). This project's own DeepFace errors (see Evaluation, Stage 1 results) came from a single tester, so nothing can be assumed about how the model behaves for users with different skin tones or genders, either in how often it fails or in which way. Whisper and faster-whisper show the same pattern for speech: word error rates rise measurably for non-native and non-American accents compared with American and Canadian speakers (Graham & Roll, 2024). A user's accent could therefore quietly degrade the transcript that feeds every later stage, without the user ever being told. Finally, the terminal and OpenCV interface (see Session flow and user interaction) was chosen for development speed rather than accessibility. It has no screen-reader support, and the picture-in-picture display makes no accommodation for colour-blindness or low vision, so a user who cannot read the terminal clearly cannot use the tool at all. None of these three limitations was addressed in this build.

Ethical safeguards, privacy management and crisis boundary

Because the prototype works in an area close to mental health, the design treats safety as a fixed boundary in the code rather than something left to the language model. The safeguards built in are: consent before anything runs, a clear statement that the tool does not diagnose, a seek-help assessment computed in code from the user's own words rather than by the language model, contact details supplied only by fixed code rather than generated text, processing kept on the device, consent recorded for each session, and signposting to professional support at the end of every session. These measures follow WHO guidance that AI used in health settings should protect autonomy, privacy, safety, transparency and inclusiveness (World Health Organization, 2021).

The prototype is not suitable for handling crises, nor for unsupervised use by vulnerable people going through severe mental-health difficulties. The seek-help flag is derived from a general text-emotion model, which detects emotional tone rather than intent: it can misread a message, for example classifying "I feel overwhelmed lately" as surprise, and the current evaluation gives no evidence that it can reliably spot suicidal intent or other emergencies. The closing resources are therefore a signpost, not a way of detecting a crisis. A real product would need a separately tested safety pathway, explicit crisis messaging, a route to a human, or emergency-service guidance suited to the user's location, along with testing on realistic safety cases, before it could claim to support anyone in crisis.

Privacy also means more than keeping data off the network. Nothing is sent to the cloud, but the system does write sensitive session logs to the local device, so privacy still depends on who can access that device and on how long the logs are kept. The current prototype gives the user no retention period and no way to delete their data. A future version should show what is stored, where it is stored and for how long, and give the user a way to delete it immediately.

Structure of the project

Implemented guided session flow

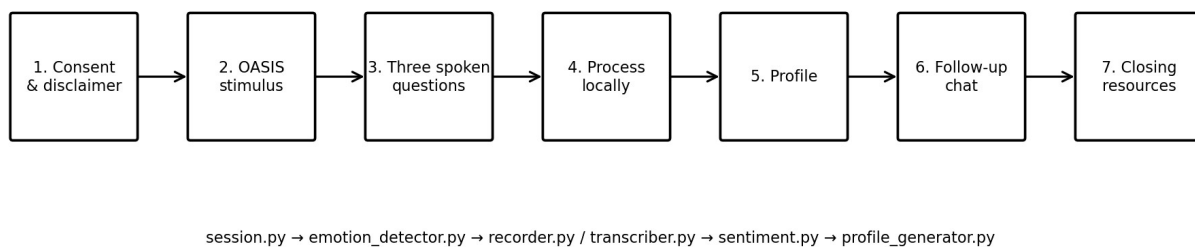


Figure 1. Implemented seven-step user session flow, from consent through stimulus, spoken responses, local processing, profile generation, follow-up chat and closing resources.

The user sits in front of their laptop and watches a short sequence of emotionally provocative images. While this happens, the webcam captures live video with the user's consent and feeds it to a facial emotion recognition model (DeepFace), which samples frames in real time and classifies each into one of seven emotion categories. The complete interaction sequence is summarised in Figure 1.

The system then asks the user three questions, which they answer aloud. Their voice is transcribed by Whisper (faster-whisper). The resulting text is passed to a sentiment analysis model, which identifies the emotional tone behind what the user said.

The collected signals are passed to a local language model (Ollama), which writes a personalised, non-diagnostic reflection with wellbeing suggestions, then opens a follow-up conversation. The session closes with professional-support resources, adding a direct recommendation if the user's own words indicate distress.

4. Implementation – word count (1281)

Orchestration overview

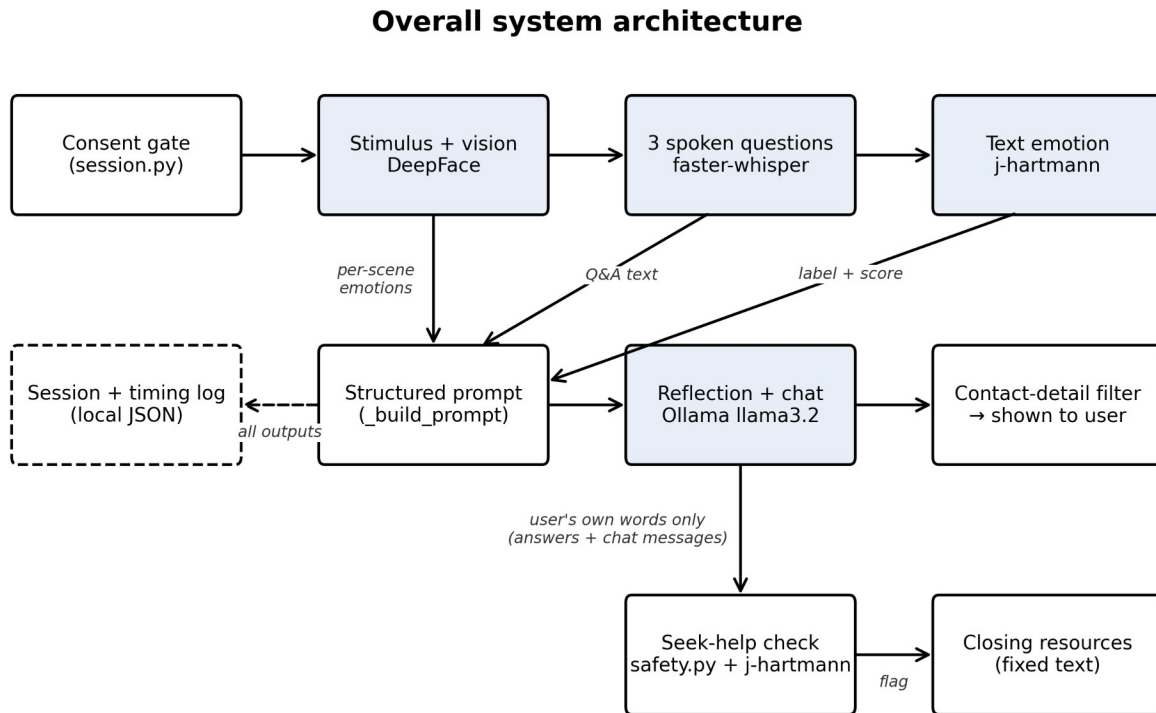


Figure 2. Overall architecture. Pre-trained models (shaded) are orchestrated by `pipeline.py`; consent, the contact-detail filter, the seek-help check and the closing resources run in deterministic code.

A `pipeline.py` module acts as the orchestrator. Each model lives in its own module with a clean interface (a class or a run function returning a small structured result), and `pipeline.py` is solely responsible for sequencing the stages and passing data between them. This keeps each model self-contained and independently testable, and allows stages to be added without changing the modules already written. Figure 2 summarises the resulting architecture and data flow.

Each completed run is saved as a timestamped JSON session log containing consent status, demo status, stage outputs, the three question-and-answer pairs, the generated profile and chat history. This makes each session auditable and distinguishes real participant sessions from demonstrations.

Stage 1: Facial emotion recognition (DeepFace)

DeepFace (Serengil & Ozpinar, 2021) performs facial-emotion classification from webcam frames. Each sampled frame is analysed independently and mapped to one of seven emotion labels; the lack of temporal context is relevant to the direction-inconsistent misclassifications reported in Evaluation.

The vision stage exposes `run_session()` and a separate `summarise_timeline()` function, allowing the deterministic summarisation logic to be tested without webcam hardware. Figures 4 and 5 show the earlier single-window build used to confirm that DeepFace ran at an acceptable frame rate on the target hardware before the stimulus display was added.

OASIS (Kurdi et al., 2017) replaced the placeholder scenes shown in Figure 3. Its normative valence and arousal ratings support deliberate selection of stimuli across threatening, joyful, sad and relaxing conditions. IAPS was considered but was not obtainable through an official route at the time of development, so OASIS was selected as the practical and openly available alternative.

```
# Simulated video sequence (start, end, label).
# Placeholder stimulus for the prototype
video_sequence = [
    (0, 5, "threatening scene"),
    (5, 10, "joyful scene"),
    (10, 15, "sad scene"),
    (15, 20, "relaxing scene"),
]

def get_current_scene(elapsed):
    """Return the scene label active at the given elapsed time, or 'end'."""
    for start, end, label in video_sequence:
        if start <= elapsed < end:
            return label
    return "end"
```

Figure 3. The placeholder stimulus schedule used in the feature prototype: four hard-coded scene labels on fixed timestamps, with `get_current_scene()` returning whichever label is active. This was replaced by the OASIS-driven schedule described above.



Figure 4. An early single-window build, before the picture-in-picture layout, with `DEBUG_OVERLAY` enabled. The frame is labelled `sad` during the relaxing scene - an instance of the direction-inconsistent misclassification of the same kind as those reported in Evaluation, Stage 1 results.

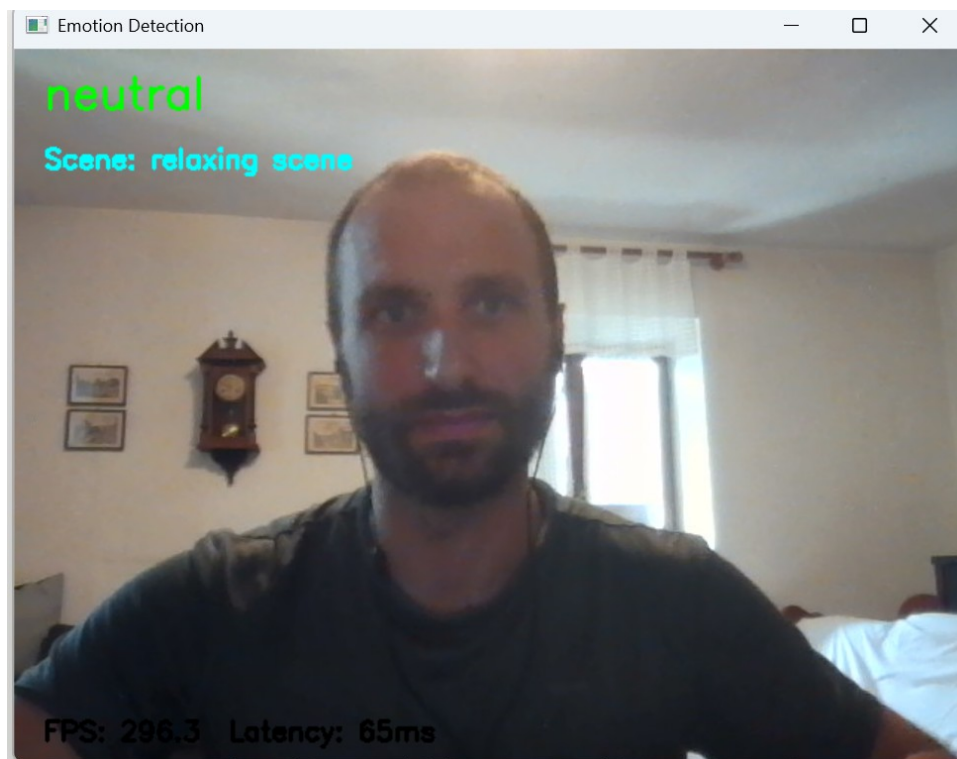


Figure 5. The same build with the FPS and per-analysis latency counters used to confirm that frame capture remains fluid while DeepFace is sampled every tenth frame on CPU-only hardware.

OASIS integration and picture-in-picture display

The final display uses a picture-in-picture layout: the stimulus occupies the main window and the webcam feed appears as a small inset. Figure 6 shows the resulting window. A `DEBUG_OVERLAY` flag exposes scene/emotion labels for development but remains disabled during real sessions so participants are not prompted by the system's own interpretation.



Figure 6. The running session with a real OASIS stimulus filling the window and the live webcam feed inset bottom-right, shown here with `DEBUG_OVERLAY` enabled so the active scene label, the live detected emotion, and the FPS/latency counters are visible. All three overlays are hidden by default during a real session.

Stage 2: Speech-to-text (Whisper / faster-whisper)

The project uses faster-whisper rather than the original openai-whisper, since it is reported to run up to four times faster at equivalent accuracy (about 2x in this project's own benchmark) and supports int8 quantisation to reduce memory use - both matter given the project runs locally with no GPU. The component is built as a Transcriber class: the model is loaded once in `__init__` and reused for every subsequent call, avoiding the cost of reloading it each time.

Stage 3: Sentiment / text-emotion analysis

The text stage is implemented in `sentiment.py`, following the same structural pattern as the Transcriber class: a `SentimentAnalyzer` class loads the chosen model once at construction and exposes a single `analyze(text)` method that returns a small structured result (an `EmotionResult` namedtuple of label and confidence score), so the rest of the pipeline always receives a predictable shape regardless of which model sits underneath.

`j-hartmann/emotion-english-distilroberta-base` was selected after benchmarking it against VADER and `tabularisai/multilingual-sentiment-analysis` on a fixed set of test transcripts (see Evaluation, Stage 3 results, for the full comparison). Its key advantage is sharing DeepFace's seven-emotion label space, so facial and verbal emotion can later be compared directly by the language model rather than reconciled across two different vocabularies.

Stage 4: Local language model (Ollama)

The language stage is implemented in `profile_generator.py` as a `ProfileGenerator` class that wraps a local Ollama model. A fixed system prompt establishes the scope boundary in the code rather than only in the report: it states that the assistant is not a therapist, forbids diagnosing any medical or psychiatric condition, restricts the output to reflecting observed patterns and general wellbeing practices, and requires a clear recommendation to speak to a mental health professional where the observed patterns suggest significant or persistent distress.

The prompt also instructs the model never to refuse when a user expresses distress, but to acknowledge it warmly and encourage contact with someone they trust or a professional, and never to state phone numbers, hotline names or websites itself. Because a 3B model does not follow such instructions reliably, `strip_contact_details()` additionally removes any reply line containing a phone number, SMS short code, URL or template placeholder before it is displayed or stored (Listing 1); verified resources are printed only by the session layer.

```

55 # Contact-detail filter. In testing, llama3.2 kept inventing hotline numbers
56 # (including an outdated US number) even when the system prompt told it not
57 # to, which are wrong or useless for a European user. Any line of a reply
58 # containing a phone number, an SMS short code, a URL, or a placeholder like
59 # "[insert ... number]" is removed in code before it is shown or stored. The
60 # verified resources are printed by session.print_closing() instead.
61 _CONTACT_PATTERN = re.compile(
62     r"(\+?\d[\d\s()]{5,}\d)"          # phone numbers, e.g. 1-800-273-8255
63     r"|\b\d{5,}\b)"                  # SMS short codes, e.g. 741741
64     r"|(https?:\/\/\S+|www\.\S+)"      # URLs
65     r"|\[insert[^\]]*\])",            # template placeholders
66     re.IGNORECASE,
67 )
68
69 def strip_contact_details(text):
70     """Remove every line of text that contains contact details."""
71     kept = [line for line in text.splitlines() if not _CONTACT_PATTERN.search(line)]
72     return "\n".join(kept).strip()
73

```

Listing 1. The contact-detail filter in profile_generator.py: any reply line containing a phone number, SMS short code, URL or template placeholder is removed before it is shown or stored.

A private `_build_prompt()` method assembles the three upstream stages into one message: the dominant facial emotion and reading count per scene, the combined question-and-answer transcript, and the text-emotion label with its confidence score. Presenting all three signals in a single structured prompt is what allows the model to comment on a mismatch between expression and speech, which is the reflective content no single stage can produce on its own.

`generate_profile()` returns a `Profile` namedtuple of the reply text and raw text; the `seek_help` field is not set by the language model. It is assessed at the end of the session by `assess_session()` in `safety.py`, which scores only the user's own words (the three spoken answers and every chat message, never the model's replies) with the j-hartmann text-emotion model. The flag is set if any single text is classified as sadness, fear, anger or disgust with confidence ≥ 0.90 , or if at least two texts are negative with confidence ≥ 0.70 (Listing 2). The assessment runs in plain Python after the chat, so the model cannot set, suppress or contradict it, and each flag is logged with the per-text scores and the rule that triggered it. The redesign that led to this rule is reported in Evaluation.

```

77 def assess_session(user_texts, analyzer, emotion_summary):
78     """Assess the whole session at its end.
79
80     user_texts: list of strings written or spoken by the user only.
81     analyzer:   object with .analyze(text) -> EmotionResult(label, score).
82     emotion_summary: {scene: (dominant_emotion, readings)} from DeepFace.
83     """
84     texts = [t for t in user_texts if t and t.strip()]
85     scored = []
86     for text in texts:
87         result = analyzer.analyze(text)
88         scored.append({"text": text, "label": result.label, "score": result.score,
89                       "distress": is_distress(result),
90                       "strong": is_strong_distress(result)})
91
92     distress_count = sum(s["distress"] for s in scored)
93     strong_count = sum(s["strong"] for s in scored)
94     share = (distress_count / len(scored)) if scored else 0.0
95     face_ratio = negative_scene_ratio(emotion_summary)
96
97     flag = strong_count >= 1 or distress_count >= MIN_DISTRESS_TEXTS
98
99     reason = (f"{strong_count} text(s) negative >= {STRONG_CONFIDENCE_THRESHOLD:.2f} "
100             f"(1 needed); {distress_count} of {len(scored)} texts negative >= "
101             f"{TEXT_CONFIDENCE_THRESHOLD:.2f} ({MIN_DISTRESS_TEXTS} needed); "
102             f"negative face in {face_ratio:.0%} of scenes "
103             f"(logged only, not used for the flag)")
104     return RiskAssessment(flag, reason, round(share, 3), round(face_ratio, 3), scored)
105

```

*Listing 2. `assess_session()` in `safety.py`. Each of the user's own texts is scored with *j-hartmann*; the flag is set if one text is negative (sadness, fear, anger or disgust) with confidence ≥ 0.90 , or at least two are negative with confidence ≥ 0.70 .*

`chat()` continues the same conversation, appending each user message and reply to a history list held on the instance, so the follow-up conversation retains the profile the model has just written as context rather than starting from an empty state on every turn. The *ollama* package is imported inside `__init__` rather than at module level, matching the lazy-import pattern used for *DeepFace*, *faster-whisper* and *transformers*, so every module remains importable - and unit-testable - on a machine where the heavy dependencies are not installed.

Integration test

`run_pipeline()` in `pipeline.py` sequences the four stages and is the only place where data crosses between them. It records the wall-clock duration of each stage into a dictionary as it runs, then writes two artefacts on completion: a timestamped session log holding consent status, demo status, the three question-and-answer pairs, every stage output and the full chat transcript, and a separate timing log holding per-stage seconds with totals reported both including and excluding the user-paced chat. After the chat ends, the user's spoken answers and chat messages are passed to the seek-help assessment, whose result drives the closing message and is written to the session log (Listing 3).

```

228 chat_messages = run_chat_loop(generator)
229 stage_timings["chat_loop"] = time.perf_counter() - t0
230
231 # End-of-session seek-help assessment: user's own words only (spoken
232 # answers + chat messages).
233 t0 = time.perf_counter()
234 user_texts = [pair["answer"] for pair in qa_pairs] + chat_messages
235 risk = assess_session(user_texts, analyzer, summary)
236 profile = profile._replace(seek_help=risk.flag)
237 stage_timings["seek_help_assessment"] = time.perf_counter() - t0
238
239 session.print_closing(profile)
240
241 save_session_log(summary, qa_pairs, text_emotion, profile, generator, use_sample, risk)
242 save_timing_log(stage_timings)

```

Listing 3. The end of `run_pipeline()` in `pipeline.py`: the user's spoken answers and chat messages are collected, assessed by `safety.py`, and the result drives the closing message and the session log.

A `--demo` flag substitutes the bundled sample audio for live microphone capture, skips the webcam stimulus in favour of a fixed per-scene emotion summary, and auto-confirms consent, which allows the full pipeline to be demonstrated and tested quickly on any machine; the flag is recorded in the session log so demonstration runs are never mistaken for participant sessions.

Session flow and user interaction

The four model stages are wrapped in a guided session implemented in `session.py`, so that what a user encounters is a structured, consent-first experience rather than the developer pipeline the stages alone provide. The flow runs in seven steps: consent and disclaimer, the stimulus sequence with live facial analysis, three spoken questions, processing, the generated profile, the follow-up conversation, and a closing message pointing to professional support.

Consent is handled by `get_consent()`, which prints a disclaimer stating what will be captured (webcam and microphone), what the system will not do (diagnose, or transmit anything off the machine), that the session is written to a local log file, and how to stop at any point. The check fails closed: only an explicit "yes" proceeds, while a blank line, a refusal, an end-of-file or a keyboard interrupt are all treated as consent withheld. Figure 7 shows the screen presented to the user. Where consent is not given, `run_pipeline()` returns before any stage executes, so nothing is captured, recorded or logged for a declined session.

```

=== Consent ===

=====
Emotional AI Mirror - before we start
=====
This session will:
- turn on your webcam and analyse your facial expressions
  while you watch a short sequence of images
- record your microphone while you answer three spoken
  questions, and transcribe what you say
- use both to generate a short, non-clinical reflection on
  your emotional patterns, which you can then discuss

Please also note:
- this tool does NOT diagnose any medical or psychiatric
  condition, and is not a replacement for professional care
- all processing happens locally on this machine - nothing
  is uploaded or sent to a server
- your recording and the session transcript are saved to a
  local log file on this machine so the session can be
  reviewed later; nothing leaves this device
- you can stop at any point by closing the webcam window or
  pressing Ctrl+C

You must be 18 or older to use this tool.

=====
Type 'yes' to continue, or press Enter to exit: █

```

Figure 7. The consent and disclaimer screen shown before any capture begins, stating what is recorded, that the tool does not diagnose, that processing and storage are local, and that the session can be stopped at any point.

```

},
"spoken_qa": [
  {
    "question": "How are you feeling right now?",
    "answer": "1,2,3 test test. I'm feeling really good. I'm feeling really good. 1,2,3 test."
  },
  {
    "question": "What did those images bring to mind?",
    "answer": "Well, some of them were interesting, some of them were like sad, some of them were joyful, so a bunch of different emotions."
  },
  {
    "question": "Is anything weighing on you?",
    "answer": "Yeah, I didn't like the kids crying, obviously."
  }
]
}

```

Figure 8. The spoken_qa field of a session log, showing each of the three questions stored alongside the answer it prompted rather than as one undifferentiated transcript.

The session closes with `print_closing()`, which always displays signposting to professional support and adds a more direct prompt when the end-of-session assessment in `safety.py` sets the `seek_help` flag.

```
"timestamp": "2026-07-31T14:44:38.421643+00:00",
"consent_given": true,
"demo_mode": false,
"emotion_summary": {
  "threatening scene": {
    "emotion": "sad",
    "readings": 11
  },
  "joyful scene": {
    "emotion": "sad",
    "readings": 27
  },
  "sad scene": {
    "emotion": "sad",
    "readings": 27
  },
  "relaxing scene": {
    "emotion": "sad",
    "readings": 12
  }
}
```

Figure 9. The head of a session log, showing the consent_given and demo_mode flags recorded per session alongside the per-scene emotion summary - evidence that consent is captured per run rather than assumed.

5. Evaluation – word count (1982)

Testing methodology

Evaluation uses three complementary levels: isolated model benchmarking, end-to-end integration testing, and user testing. The first stage benchmarks each model in isolation. At system level, the key metrics are completion rate, safety-response consistency, latency and user-rated outcomes. Because the prototype uses pre-trained models and the pilot contains only five convenience-sampled participants, statistical analysis is intentionally descriptive rather than inferential.

The second stage tests how the models connect: whether output from one stage reaches the next in a usable form (e.g. Whisper's transcription passing cleanly to the sentiment model), watching for mismatched data formats, empty or dropped inputs, and total pipeline processing time.

The third stage tests the experience with real users, rating perceived accuracy, clarity and willingness to use the system again on a 1-5 Likert scale. A five-participant pilot of this instrument was completed for this report and is reported below.

Stage 1 results: Facial emotion recognition

One full OASIS sequence was recorded and every per-frame reading logged against the scene active at that moment. Figure 10 plots the result. Readings were dominated by neutral, but two direction-inconsistent errors are visible: a sustained block of happy through the first half of the sad scene, and two isolated fear readings during the relaxing scene. Because each frame is classified independently, a single ambiguous expression can flip the label without any change in what the participant reports feeling.

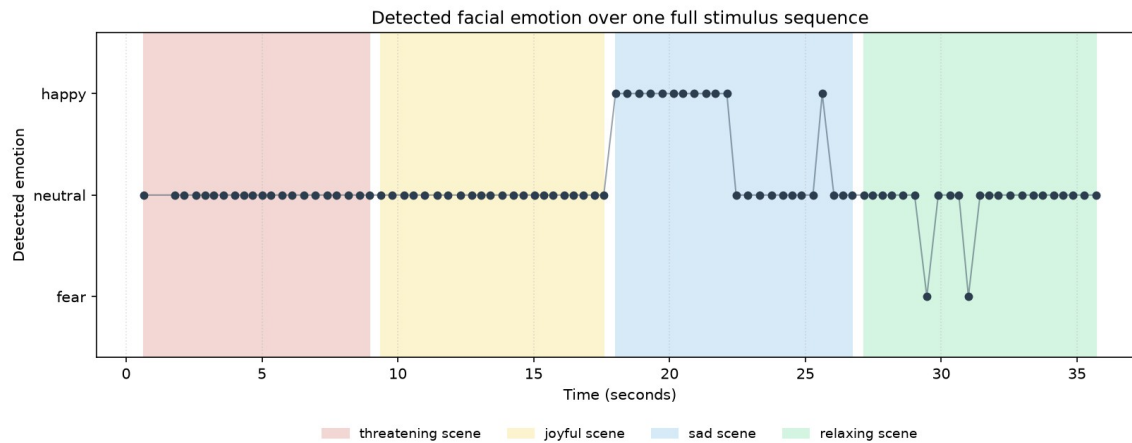


Figure 10. Detected emotion for every reading across one full OASIS sequence, with the four scenes shaded behind. The run is dominated by neutral readings, with a sustained block of happy misclassification through the first half of the sad scene - the direction-inconsistent error described above - and two isolated fear readings during the relaxing scene.

DeepFace vs. FER comparison

DeepFace was benchmarked against FER (Shenk, 2021), a separate Keras CNN emotion model with, by default, a Haar Cascade face detector rather than DeepFace's, using `emotion_benchmark.py`, sampling one frame per second through both models simultaneously from a live webcam feed. Two runs were carried out.

The first run was not tightly controlled for camera framing (the participant was not consistently facing the camera throughout). Table 2 shows the result: FER failed to detect a face on 7 of 12 sampled frames, against zero missed detections for DeepFace, while both models agreed on every frame where FER did detect a face. This is not treated as a reliable finding about the two models' relative robustness, since it may reflect FER's detector being more sensitive to off-centre framing rather than a genuine reliability gap.

Elapsed (s)	DeepFace label (latency)	FER label (latency)	Agreement
1.0	happy (425.3ms)	no face detected (14.1ms)	-
4.0	happy (70.5ms)	no face detected (13.3ms)	-
5.0	happy (69.7ms)	no face detected (13.2ms)	-
6.1	happy (65.1ms)	no face detected (13.2ms)	-
7.1	happy (63.6ms)	no face detected (13.2ms)	-
8.1	happy (72.3ms)	no face detected (11.1ms)	-
9.1	neutral (68.7ms)	neutral (17.1ms)	yes
10.1	neutral (73.9ms)	neutral (13.7ms)	yes
11.2	neutral (78.1ms)	neutral (15.5ms)	yes
12.2	neutral (75.0ms)	neutral (12.5ms)	yes
13.2	happy (63.4ms)	happy (15.3ms)	yes
14.2	happy (71.8ms)	no face detected (12.9ms)	-

Table 2. DeepFace vs. FER, run 1 (inconsistent camera framing). Average latency: DeepFace 99.8ms, FER 13.8ms. FER detected no face on 7/12 frames.

Elapsed (s)	DeepFace label (latency)	FER label (latency)	Agreement
1.0	neutral (346.5ms)	neutral (16.5ms)	yes
3.6	neutral (73.3ms)	neutral (16.4ms)	yes
4.6	neutral (74.8ms)	neutral (14.9ms)	yes
5.7	neutral (70.3ms)	neutral (16.0ms)	yes
6.7	neutral (71.5ms)	neutral (13.8ms)	yes
7.7	neutral (67.3ms)	neutral (15.1ms)	yes
8.7	neutral (66.3ms)	neutral (15.6ms)	yes
9.7	neutral (67.8ms)	neutral (14.0ms)	yes
10.7	neutral (69.2ms)	neutral (15.4ms)	yes
11.7	neutral (70.6ms)	neutral (15.0ms)	yes
12.8	neutral (71.2ms)	neutral (13.7ms)	yes
13.8	neutral (66.0ms)	neutral (15.4ms)	yes
14.8	neutral (72.8ms)	neutral (14.5ms)	yes

Table 3. DeepFace vs. FER, run 2 (controlled framing, resting face throughout). Average latency: DeepFace 91.4ms, FER 15.1ms. 13/13 frames agreed.

Run 2 (Table 3) repeated the comparison with the participant facing the camera throughout and holding a resting expression. Both models returned neutral on all 13 sampled frames, agreeing on every one, and FER missed no faces. DeepFace averaged 91.4ms per frame against FER's 15.1ms, so FER is roughly four times faster; DeepFace was nonetheless retained because it detected a face in every frame across both runs and already shares its seven-label vocabulary with the selected text-emotion model. The run also matters as a negative result: under controlled framing the neutral-to-fear/disgust drift seen in earlier informal testing did not reproduce in either model, which suggests that framing and lighting, rather than a property specific to DeepFace, account for a substantial part of the observed instability.

Stage 2 results: Speech-to-text

Both Whisper implementations were run on the same audio file at the tiny model size (Table 4). faster-whisper transcribed the test sentence correctly and completed transcription in roughly half the time of openai-whisper, at a fraction of the peak Python memory; openai-whisper loaded marginally faster but mis-transcribed one word and additionally required a system-level ffmpeg installation. faster-whisper was therefore selected, confirming a clear speed advantage on this hardware, though about 2x rather than the up to 4x reported by the developers.

Model	Model load time	Transcripti on time	Peak memory	Notes
faster-whisper (tiny)	0.74s	0.54s	13.4 MB	Transcribed the test sentence correctly in full
openai-whisper (tiny)	0.44s	1.05s	72.1 MB	Mis-transcribed one word ("microphone" -> "Micah Cup"); required a separate system-level ffmpeg install to run

Table 4. faster-whisper vs openai-whisper on an identical audio file, tiny model size, on an Intel Core i7-1185G7, 16GB RAM, no dedicated GPU. Peak memory measured with tracemalloc, isolated per model; this captures Python-level allocations only and likely understates true peak memory, since it does not see inside PyTorch's C/C++ tensor backend.

Stage 3 results: Sentiment / text-emotion analysis

Three candidate models were benchmarked on the same 10 test transcripts (see test_transcripts.py): VADER, a rule-based sentiment analyser (Hutto & Gilbert, 2014); j-hartmann/emotion-english-distilroberta-base, a transformer trained on the same seven-emotion label space as DeepFace; and tabularisai/multilingual-sentiment-analysis, a 5-class sentiment model.

j-hartmann was selected. Against the expected labels shown in Table 5, it produced 7/10 exact categorical matches (70%). This should not be interpreted as a validated emotion-recognition accuracy: the test set contains only ten hand-labelled transcripts, one expected class is mixed emotion while the model is single-label, and several classes have only one example. The result therefore helps choose between models, but says nothing about how the model would perform in general. It also uses the same seven labels as DeepFace, which makes it possible to compare the two later on, even though what those labels really mean psychologically is still uncertain.

Macro recall over the eight expected categories was 62.5%: sadness, anger and mixed emotion were not recovered, which a single accuracy figure would conceal.

Text	Expected	VADER	j-hartmann	tabularisa
Honestly I feel really good today...	happy	positive	joy	Positive
I don't know, I've just been feeling really down...	sad	negative	joy	Negative
It was a normal day, nothing much happened.	neutral	neutral	neutral	Neutral
I keep worrying about this presentation...	fear	negative	fear	Negative
It's just so frustrating, nobody ever listens...	anger	negative	sadness	Negative
I'm excited about the new job but also kind of scared...	mixed	negative	fear	Negative
I had this weird dream last night...	fear	negative	fear	Negative
Wait, I did not expect that at all, that's wild.	surprise	neutral	surprise	Negative
That really put me off, I couldn't stand it.	disgust	neutral	disgust	Negative
I'm fine.	neutral	positive	neutral	Positive

Table 5. Predicted label per model against the expected label for each test transcript, run on an Intel Core i7-1185G7, 16GB RAM, no dedicated GPU.

Table 6 summarises average latency and memory per candidate model across the same 10 test transcripts.

Model	Avg latency (ms/input)	Memory	Notes
VADER	0.18	Negligible (rule-based, no model weights)	Fastest by far, but only 3-class (positive/negative/neutral); cannot detect surprise or disgust and missed the mixed-emotion case entirely
j-hartmann/emotion-english-distilroberta-base	44.31	See note below table	7/10 correct; shares DeepFace's 7-emotion label space; only model to correctly catch surprise and disgust
tabularisai/multilingual-sentiment-analysis	30.87	See note below table	Heavily biased toward "Negative" - 8/10 predictions were Negative, including clearly positive/neutral cases

Table 6. Average latency per candidate model on an Intel Core i7-1185G7, 16GB RAM, no dedicated GPU. Memory: tracemalloc reported 193.2MB peak Python memory across the full benchmark run (all three models); per-model isolation was not measured in this run and is noted as a limitation.

Stage 4 results: Local language model (Ollama)

Integration was confirmed by running complete sessions end to end and inspecting the resulting logs. The language stage received the combined Stage 1-3 output and returned a reflection that referred to the specific scenes and spoken content of that session, and the

session log recorded each stage's output, including the end-of-session seek-help assessment with its per-text scores (Figure 11).

```

46 "seek_help_assessment": {
47   "flag": true,
48   "reason": "2 text(s) negative >= 0.90 (1 needed); 2 of 6 texts negative >= 0.70 (2 needed); negative face in 25% of scenes (logged only, not used for the flag)",
49   "distress_share": 0.333,
50   "negative_face_ratio": 0.25,
51   "scored_texts": [
52     {
53       "text": "I feel pretty sad and I feel like I need some help from maybe a professional, but I don't know, I would like to have your opinion on this.",
54       "label": "sadness",
55       "score": 0.984,
56       "distress": true,
57       "strong": true
58     },
59     {
60       "text": "Not much, some of them were very sad, and I could relate to them, but the majority were just normal, plain, neutral, that's how they felt to me.",
61       "label": "sadness",
62       "score": 0.904,
63       "distress": true,
64       "strong": true
65     },
66     {
67       "text": "Yeah, I feel like I'm overworking and also my family situation is not very good, so I need to talk to someone I think.",
68       "label": "sadness",
69       "score": 0.504,
70       "distress": false,
71       "strong": false
72     },
73     {
74       "text": "I feel overwhelmed lately",
75       "label": "surprise",
76       "score": 0.756,
77       "distress": false,
78       "strong": false
79     },
80   ]

```

Figure 11. The `seek_help_assessment` block of a live session log: the flag, the rule that set it, and each user text with its emotion label and score. Two texts scored sadness ≥ 0.90 , so the flag was set.

During integration testing, llama3.2 once answered an ambiguous follow-up with an unrelated safety refusal, a risk in a mental-health-adjacent tool where an erratic reply can feel dismissive.

A structured robustness test (`chat_robustness_test.py`) quantified this: 10 varied and ambiguous follow-up messages-spanning short/terse input, an off-topic question, ambiguous emotional statements, a direct diagnosis request, sarcasm, a meta-question about data handling, nonsense input, a topic shift, mild distress language, and a reference back to earlier context-were sent through the same conversation history on both llama3.2 and qwen2.5:3b, starting from an identical profile context on each model. Table 7 summarises the outcome per model.

Model	On-topic / appropriate	Inappropriate refusal	Factually wrong (system-inconsistent)
llama3.2	7/10	2/10	1/10
qwen2.5:3b	9/10	0/10	1/10

Table 7. Chat robustness test, 10 messages per model, classified from the full reply transcripts (generated by `scripts/chat_robustness_test.py`).

The chat test gives a higher-level robustness measure than a single successful conversation: llama3.2 produced appropriate/on-topic responses for 7/10 prompts (70%), with 2/10 inappropriate refusals and 1/10 system-inconsistent factual error; qwen2.5:3b produced 9/10 appropriate responses (90%), no inappropriate refusals and 1/10 system-inconsistent

factual error. The latter error is especially important because both models incorrectly stated that conversation data were not saved, while the application actually writes session logs locally. This means safety-critical system facts should be stated by fixed application-level text rather than delegated to the language model.

Model	Time to first token	Total generation time	Approx. size	Output quality (notes)
llama3.2	10.46s	28.04s	~2GB	1332-character reply; explicitly recommended professional support given the fear signal
qwen2.5:3b	8.79s	18.99s	~2GB	789-character reply, more concise; covered the same emotional patterns but did not recommend professional support for a comparable distress signal

Table 8. Local LLM comparison for profile generation, run on an identical sample emotion summary (facial emotions + transcript + sentiment scores) on an Intel Core i7-1185G7, 16GB RAM, no dedicated GPU.

qwen2.5:3b was 16% faster to first token and 32% faster overall, but omitted the professional-help recommendation. Since the redesign, seek-help runs in code, so this no longer affects safety, and qwen2.5:3b also did better in the chat test (Table 7). llama3.2 was kept because the rest of the evaluation used it; testing a switch is the first recommended extension.

End-to-end pipeline timing

Run	Vision (stimulus)	Spoken Q + transcription	Text-emotion	Profile generation	Total (excl. chat)
1	41.48s	26.41s	2.41s	46.97s	117.27s
2	49.45s	20.92s	13.81s	34.57s	118.74s
3	47.48s	57.70s	10.34s	32.23s	147.75s
Average	46.1s	35.0s	8.9s	37.9s	~128s

Table 9. End-to-end pipeline timing across three full live runs (consent through profile generation, excluding the open-ended follow-up chat), on an Intel Core i7-1185G7, 16GB RAM, no dedicated GPU.

Table 9 reports three complete live runs, timed per stage. Chat is excluded from the reported total because it is user-paced and open-ended rather than a fixed system cost; including it (run 1: 138.13s, run 2: 144.52s, run 3: 194.50s) would conflate the pipeline's own processing time with however long the user chooses to keep talking. The consistent finding across all three runs is that the system takes just over two minutes, on average around 128 seconds, from the start of the stimulus to a delivered profile, with no cloud round-trip anywhere in

that time. Profile generation (Ollama, average 37.9s) and the vision stage (average 46.1s, which includes the 36-second stimulus display time (twelve images at three seconds each) rather than being pure DeepFace inference cost) are the two largest and most variable contributors; text-emotion analysis is 2.4-13.8s, the higher end likely reflecting model warm-up on first call rather than a real per-session cost. Figure 12 shows a timing log as written at the end of a run. This is the latency cost of local processing argued for in Design.

```
1  {
2    "timestamp": "2026-08-01T07:44:01.887016+00:00",
3    "stage_seconds": {
4      "stimulus_and_vision": 47.483,
5      "spoken_questions_and_transcription": 57.698,
6      "text_emotion_analysis": 10.336,
7      "profile_generation": 32.23,
8      "chat_loop": 46.753
9    },
10   "total_seconds_excl_chat": 147.748,
11   "total_seconds_incl_chat": 194.502
12 }
```

Figure 12. A timing log written at the end of a completed session, recording per-stage seconds and both totals. This is the raw form of the run 3 column in Table 9.

Session logging

Every completed run writes a timestamped JSON log recording whether consent was given, whether the run was a demo, the three question-and-answer pairs, the output of each stage, the generated reflection and the full chat transcript (Figures 8, 9 and 13). This means each session can be checked afterwards, demo runs stay clearly separate from participant runs, and the evidence used throughout this chapter came straight from the logs rather than being copied by hand from the terminal.

```

},
"chat_history": [
  {
    "role": "user",
    "content": "Here is what was observed during the session:\n\nFacial emotions p
  },
  {
    "role": "assistant",
    "content": "Here's my reflection:\n\nI notice that despite expressing joy and
  },
  {
    "role": "user",
    "content": "hey there, nice talking to you"
  },
  {
    "role": "assistant",
    "content": "It was great chatting with you too! I'm glad we could have a conver
  }
]
}

```

Figure 13. The `chat_history` field of a session log: the reflection and the follow-up exchange are stored as one conversation, so later replies keep the profile as context.

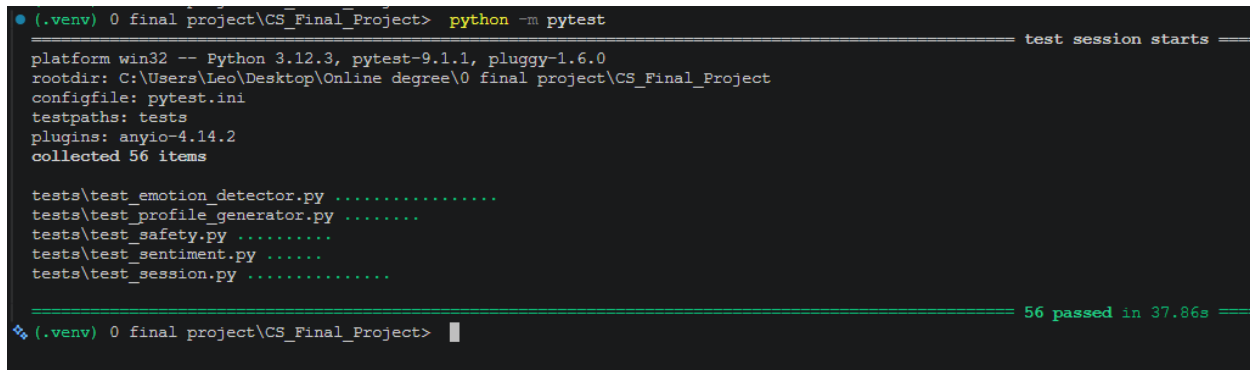
Unit testing

Module	Tests	Main coverage
Vision	17	Dynamic stimulus scheduling, scene/stimulus lookup, positioning
Sentiment	6	Empty input, result extraction, model initialisation
Profile generator	8	Prompt assembly, contact-detail filter, chat history
Session flow	15	Consent, questions, demo mode, closing resources
Safety (seek-help)	10	Flag rules, thresholds, per-text logging, regression session

Table 10. Unit-test coverage by module. Tests mock hardware and external model services so the suite isolates project-owned deterministic logic.

A pytest suite covers deterministic, hardware-independent logic across the five testable modules, using mocks so tests run offline without real models or hardware (Table 10). Vision tests cover dynamic OASIS scheduling and scene/stimulus lookup; sentiment tests cover

empty-input handling and model-result extraction; profile-generator tests cover prompt construction, the contact-detail filter and conversation state; safety tests cover the seek-help rule, its thresholds and a regression case from a real session; session tests cover the fail-closed consent gate, question ordering, demo mode and closing resources. The full suite of 56 tests passes in 37.86s with no external services, hardware or downloaded models required (Figure 14), providing regression evidence for the project-owned logic rather than validating the underlying pre-trained models.



```

(.venv) 0 final project\CS_Final_Project> python -m pytest
platform win32 -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\Leo\Desktop\Online degree\0 final project\CS_Final_Project
configfile: pytest.ini
testpaths: tests
plugins: anyio-4.14.2
collected 56 items

tests\test_emotion_detector.py .....
tests\test_profile_generator.py .....
tests\test_safety.py .....
tests\test_sentiment.py .....
tests\test_session.py .....

===== 56 passed in 37.86s =====
(.venv) 0 final project\CS_Final_Project>

```

Figure 14. The full pytest suite: 56 tests across the five testable modules, passing in 37.86s with no external services, hardware or downloaded models required.

User testing

A pilot study was run with 5 participants (friends and family, convenience sample); a larger 6-10 person study was planned but not carried out within the project timeline, and is noted as a limitation. Each participant received a verbal explanation of the session and its non-diagnostic purpose, completed the session, and rated perceived accuracy, clarity and willingness to use the system again on a 1-5 scale. The sample is too small for population-level inference; its purpose was to test the interaction and the evaluation instrument. Table 11 reports the resulting scores.

Participant	Accuracy	Clarity	Would use again
Participant 1	3	2	4
Participant 2	4	5	5
Participant 3	3	4	4
Participant 4	2	3	3
Participant 5	5	5	5
Mean	3.4	3.8	4.2

Table 11. Post-session Likert scores (1-5), $n=5$ pilot participants; SD values are reported in the Evaluation text.

Willingness to use the system again scored highest (mean 4.2/5), followed by clarity (3.8/5) and perceived accuracy (3.4/5). The sample standard deviations were 0.84, 1.30 and 1.14 respectively, showing substantial individual variation despite the small sample. Two participants reported close agreement with the reflection, while one described it as too

general and another said fear was not captured well. These comments align with the model-level findings rather than establishing psychological validity.

A more important finding emerged from the session logs: all 5 sessions were flagged `seek_help: true`, despite most participants describing themselves as fine, bored, or mildly skeptical rather than distressed. This exposes a potentially serious false-positive pathway.

Seek-help redesign and re-evaluation

Inspection of the code showed the cause. The flag was set by scanning the language model's own reply for words such as "professional", while the system prompt instructs the model to state that it is not a substitute for professional care, so almost every reply matched. The flag therefore measured the model's instructions rather than the user's state: every readable session logged before the redesign (13/13) was flagged.

The flag was redesigned to use only the user's own words (spoken answers and chat messages), scored by the existing j-hartmann model, and to run in code after the chat (see Implementation, Stage 4). Because every log stores these texts, the pilot sessions could be re-assessed without new sessions using `eval_seek_help.py` (Table 12).

Sessions	Old method	New method
Pilot participants (n = 5)	5/5 flagged	0/5 flagged
Earlier development runs (n = 8)	8/8 flagged	0/8 flagged
Distress test sessions (n = 4)	–	4/4 flagged
Neutral test session (n = 1)	–	0/1 flagged

Table 12. Seek-help flag before and after the redesign, re-scored from saved session logs with `scripts/eval_seek_help.py`; logs are kept locally and not published, as they contain participants' words.

The new rule separated distressed from neutral sessions on this set, but the result is a sanity check, not a validation: there is no clinical ground truth, the pilot participants reported no distress, and the distress sessions were scripted by the developer. Per-text scores also exposed the model's limits: "I feel overwhelmed lately" was classified as surprise (0.76), and in one session a sentence describing the stimulus images as sad counted towards the flag, because the model reads emotional tone, not whom it refers to.

Participant comments also suggested two lower-priority interface changes: adding audio to the stimulus presentation and making the visual interaction more accessible. These were recorded as future iterations rather than implemented.

Inclusive design and demographic limitations

Most stage-level benchmarks were produced by one user, on one machine, in English. They therefore cannot establish accuracy across demographics, accents or hardware. The demographic biases inherited from the pre-trained models (see Design, Inclusive design) were therefore not measured.

Evaluation against project objectives

Table 13 assesses each of the four aims stated in the Introduction against the evidence in this chapter.

Aim	Outcome	Evidence and critical assessment
1. Complete consent-first local pipeline	Achieved	All four stages ran locally end to end, behind a consent gate, in every logged session, and the automated suite passed (56 tests). The cost is latency: about 128 s of processing before the conversation starts (Table 9), the price of keeping data on the device.
2. Compare candidate models with reproducible component-level measurements	Partially achieved	Every stage had a compare-and-choose benchmark, and two choices were clear: faster-whisper was faster and more accurate, and j-hartmann was the only text model to recover surprise and disgust. The measurements are too small to support accuracy claims: 10 transcripts for text, one audio file for speech, and two short webcam runs by one person for vision. DeepFace was retained over FER, which was about six times faster, mainly on the evidence of one poorly framed run. The results are sufficient for choosing components, not for claiming how well they work.
3. Evaluate end-to-end robustness, latency and user-facing behaviour	Partially achieved	Latency was measured per stage, and the chat test turned two anecdotes into counts: llama3.2 gave 2/10 inappropriate refusals, and both models misstated how data is stored. The pilot had five convenience participants and no pre-session self-report, so perceived accuracy (3.4/5) cannot be separated from politeness or novelty effects. Facial results depend largely on one participant.
4. Identify safety, privacy and validity limitations	Achieved	The evaluation found problems that a demonstration would have hidden: direction-inconsistent facial readings, a seek-help flag that fired in every pilot session, incorrect system facts from the language model, and session logs with no retention or deletion control. The seek-help problem was traced to its cause, redesigned and re-measured (Table 12). The other three are documented but not yet fixed.

Table 13. Outcome of each project aim against the evaluation evidence.

The pattern across the table is that the project met its engineering aims and fell short where a claim would require data the project did not collect. Every validity claim was limited by sample size: of benchmark inputs, of participants, and of people whose faces were tested. None was limited by the pipeline failing to run.

6. Conclusion – word count (428)

This project set out to demonstrate a fully local, privacy-preserving proof of concept for emotional self-reflection by orchestrating facial-emotion recognition, speech transcription, text-emotion analysis and a local language model. The implemented system achieves that technical objective: the four stages run through a guided, consent-first session, generate a non-diagnostic reflection, support follow-up conversation and save the session locally without cloud processing.

The implementation demonstrates that the models can be combined into a coherent pipeline rather than used as isolated demonstrations. Model selection was evidence-led: faster-whisper outperformed the original Whisper implementation on the test audio; j-hartmann achieved the strongest result among the tested text-emotion candidates while sharing DeepFace's seven-label vocabulary; DeepFace was retained over FER after controlled comparison; and llama3.2 was kept over qwen2.5:3b, which was faster and better in the chat test, because the rest of the evaluation used it. The system is also supported by 56 passing unit tests and three end-to-end timing runs.

Two lessons extend beyond this project. First, a safety signal must not be derived from the output of the model it is meant to check: the original seek-help flag measured the system prompt's instructions rather than the user, and only an independent signal computed from the user's own words separated neutral from distressed sessions. Second, small local language models did not reliably follow negative instructions such as "never give phone numbers" or "never refuse", so safety-relevant behaviour had to be enforced in code (the contact-detail filter and the fixed closing resources) rather than requested in the prompt. Both suggest that in orchestrated systems the guarantees belong in the deterministic glue between models, not in the models themselves.

The main contribution of the project is the integration and evaluation of the local AI components. The results do not support claiming that the system can accurately infer psychological state. The average processing time of roughly 128 seconds shows what keeping everything local costs the user. At the same time, the inconsistent facial-emotion readings, the small benchmark sets, the failures in the chat test and the original seek-help flag firing in every session all show that getting the components to work together does not make the output psychologically or clinically sound. The right description at this stage is a proof of concept. Before any wider or clinical use could be considered, the project would need a larger and deliberately diverse evaluation, tested safety cases, controlled testing of the assumptions behind facial-emotion classification, user control over how long data is kept and when it is deleted, and success criteria agreed in advance for the system as a whole.

References

Abby Health. (2026). Abby AI: Medical decision support system.
<https://www.abbyhealth.com>

Affectiva. (2016). The science behind Affectiva's emotion AI. <https://www.affectiva.com>

Baltrušaitis, T., Ahuja, C., & Morency, L.-P. (2019). Multimodal machine learning: A survey and taxonomy. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 41(2), 423–443. <https://doi.org/10.1109/TPAMI.2018.2798607>

Barrett, L. F., Adolphs, R., Marsella, S., Martinez, A. M., & Pollak, S. D. (2019). Emotional expressions reconsidered: Challenges to inferring emotion from human facial movements. *Psychological Science in the Public Interest*, 20(1), 1–68. <https://doi.org/10.1177/1529100619832930>

Boehner, K., DePaula, R., Dourish, P., & Sengers, P. (2007). How emotion is made and measured. *International Journal of Human-Computer Studies*, 65(4), 275–291. <https://doi.org/10.1016/j.ijhcs.2006.11.016>

Buolamwini, J., & Gebru, T. (2018). Gender shades: Intersectional accuracy disparities in commercial gender classification. *Proceedings of Machine Learning Research*, 81, 77–91. <https://proceedings.mlr.press/v81/buolamwini18a.html>

Ezell, A. (2026). Evaluating AI chatbots for mental health diagnosis and treatment planning [Unpublished study]. Widener University.

Ghatoray, S. K., & Li, Y. (2025). Automated UX insights from user research videos by integrating facial emotion and text sentiment (arXiv:2503.22510). Department of Computer Science, Brunel University London. <https://arxiv.org/abs/2503.22510>

Graham, C., & Roll, N. (2024). Evaluating OpenAI's Whisper ASR: Performance analysis across diverse accents and speaker traits. *JASA Express Letters*, 4(2), Article 025206. <https://doi.org/10.1121/10.0024876>

Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. *Proceedings of the 34th International Conference on Machine Learning, PMLR* 70, 1321–1330.

Hartmann, J. (2022). Emotion English DistilRoBERTa-base [Model]. Hugging Face. <https://huggingface.co/j-hartmann/emotion-english-distilroberta-base>

Hugging Face. (2024). Text classification. <https://huggingface.co/tasks/text-classification>

Hutto, C. J., & Gilbert, E. (2014). VADER: A parsimonious rule-based model for sentiment analysis of social media text. *Proceedings of the Eighth International AAAI Conference on Weblogs and Social Media*, 216–225.

Kurdi, B., Lozano, S., & Banaji, M. R. (2017). Introducing the Open Affective Standardized Image Set (OASIS). *Behavior Research Methods*, 49(2), 457–470. <https://doi.org/10.3758/s13428-016-0715-3>

Laidi, C., Blampain-Segar, L., Godin, O., de Danne, A., Leboyer, M., & Durand-Zaleski, I. (2023). The cost of mental health: Where do we stand in France? *European Neuropsychopharmacology*, 69, 87–95. <https://doi.org/10.1016/j.euroneuro.2023.02.004>

Lee, J., Lee, D., & Lee, J.-g. (2024). Influence of rapport and social presence with an AI psychotherapy chatbot on users' self-disclosure. *International Journal of Human-Computer Interaction*, 40(7), 1620–1631. <https://doi.org/10.1080/10447318.2022.2146227>

Meta. (2024). Llama 3.2 [Large language model]. <https://ollama.com/library/llama3.2>

Miller, R. (2017, September 13). New Affectiva cloud API helps machines understand emotions in human speech. *TechCrunch*. <https://techcrunch.com/2017/09/13/new-affectiva-cloud-api-helps-machines-understand-emotions-in-human-speech/>

Ollama. (2024). Ollama documentation. <https://ollama.com>

OpenCV. (2026). OpenCV: Open source computer vision library. <https://opencv.org>

Poria, S., Cambria, E., Bajpai, R., & Hussain, A. (2017). A review of affective computing: From unimodal analysis to multimodal fusion. *Information Fusion*, 37, 98–125. <https://doi.org/10.1016/j.inffus.2017.02.003>

Qwen Team. (2024). Qwen2.5 technical report (arXiv:2412.15115). <https://arxiv.org/abs/2412.15115>

Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (2022). Robust speech recognition via large-scale weak supervision (arXiv:2212.04356). OpenAI. <https://arxiv.org/abs/2212.04356>

Riedl, R., Hogeterp, S. A., & Reuter, M. (2024). Do patients prefer a human doctor, artificial intelligence, or a blend, and is this preference dependent on medical discipline? Empirical evidence and implications for medical practice. *Frontiers in Psychology*, 15, 1422177. <https://doi.org/10.3389/fpsyg.2024.1422177>

Serengil, S. I., & Ozpinar, A. (2021). HyperExtended LightFace: A facial attribute analysis framework. 2021 International Conference on Engineering and Emerging Technologies (ICEET), 1–4. <https://doi.org/10.1109/ICEET53442.2021.9659697>

Shenk, J. (2021). FER: Facial expression recognition [Computer software]. GitHub. <https://github.com/justinshenk/fer>

SYSTRAN. (n.d.). faster-whisper [Computer software]. GitHub. <https://github.com/SYSTRAN/faster-whisper>

tabularisai. (2024). multilingual-sentiment-analysis [Model]. Hugging Face. <https://huggingface.co/tabularisai/multilingual-sentiment-analysis>

World Health Organization. (2021). Ethics and governance of artificial intelligence for health: WHO guidance. World Health Organization. <https://www.who.int/publications/i/item/9789240029200>